

前言：我原本以为“经典五级流水线 + Cache + 分支预测”这一套只适合跑 SPEC CPU，到了 Transformer / LLM 时代应该统统过时了，GPU 上就是一坨大矩阵乘，根本用不上那些小心思。读完 ISCA 2024 的 Splitwise、ALISA 和 Pre-gated MoE 之后，我被打脸得非常彻底——课本没过时，只是战场从“单核 CPU”搬到了“GPU 集群 + KV Cache + MoE Expert”上而已。

一、IPC 已过时，HBM 成为新主角

在《计算机体系结构》课上，我们被教育的一条铁律是：

想要提升性能，就不停地提高 IPC / ILP，把流水线拉长、做乱序执行、搞更聪明的分支预测和 Cache 策略。

但在 LLM 推理里，情况完全不一样：

1. 现代 GPU（比如 A100、H100）算力疯狂增长，但 HBM 的带宽和容量增长远远跟不上。Splitwise 这篇 ISCA'24 最佳论文给了一个很扎心的数字：H100 相比 A100，算力提升约 3.4×，功耗涨了 1.75×，但内存带宽只涨了 1.6×，容量甚至基本没涨。
2. 对于一个完整的 LLM 推理请求，论文把它拆成两段：
 1. **Prefill / Prompt 阶段**：所有输入 token 一次性跑完前向，算的是大矩阵乘，**算力密集 (compute-bound)**。
 2. **Decode / Token Generation 阶段**：一个 token 一个 token 地往后生成，要反复访问之前的 KV Cache，完全被显存带宽和 KV Cache 容量卡死 (**memory-bound**)。

Splitwise 的实验甚至量化了这个“反直觉”：对 BLOOM-176B 这种大模型，一个 1500-token 的长 prompt 的 Prefill 时间，居然和只生成 6 个 token 的 Decode 阶段差不多——大部分端到端延迟都花在后面那个“一个一个 token 慢慢啃”的阶段。

这和我印象里的“算力是瓶颈”完全相反：**GPU 上的 Tensor Core 正在摸鱼，HBM 和 KV Cache 在爆炸。**

再往下看 ALISA，就更直白了：

1. 传统 KV Cache 把 Attention 的 $O(L^2)$ 算法复杂度，换成了 $O(L)$ 的内存访问和计算。看上去很赚，但代价是 **KV Cache 内存占用随序列长度线性增长**，最后 GPU 显存爆掉，只好把 KV Cache 往 CPU / 甚至磁盘上 offload，结果又被 PCIe / 网络带宽拖死。(ar5iv)

Pre-gated MoE 则从另外一个角度补了一刀：大 MoE 模型虽然在 FLOPs 上更省，但专家参数多到离谱，比如 Switch Transformer 可以有 FLOPs 等效 dense T5 的

75 倍参数量，显存根本塞不下，只能把专家放 CPU / SSD，上来第一件事就是解决“怎么把这些专家搬来搬去”的内存与带宽问题。

所以我的顿悟是：

课本上那套“榨干 ALU、提高 IPC”的思路，在 LLM 里远远不够了。真正的主角变成了：

1. 显存带宽 (Memory Bandwidth)
2. KV Cache / Expert 参数占用的显存容量
3. GPU-GPU / GPU-CPU 之间的数据搬运和调度

而更魔幻的是：我以为分支预测、Cache 替换、虚拟存储这些“老掉牙”的内容会退场，结果在这三篇论文里，它们以各种新皮肤回归了——只是对象从“指令和数据”变成了“KV Cache 和 Experts”，位置从“单核流水线”变成了“机架级 / 集群级流水线”。

二、论文深度解读

2.1 Splitwise：把 Prefill / Decode 做成“宏观异构流水线”

他们发现了什么？

Splitwise 先做了一件很“体系结构味”的事情：测。

1. 用 Azure 线上 LLM 服务的真实 trace，统计不同业务（代码补全、对话）的 prompt 长度、输出 token 分布；
2. 在 8×H100 机器上跑 BLOOM-176B、LLaMA2-70B，分别测：
 1. Prompt 阶段的吞吐/延迟/功耗/显存占用
 2. Token 阶段的吞吐/延迟/功耗/显存占用
 3. Batching 对两阶段的影响

得到几个非常关键的 Insight（论文里也是这么列的）：

1. Prompt 阶段：
 1. 算的是大批量矩阵乘，GPU 利用率高，典型的 **compute-bound**。
 2. 批太大反而会掉速，因为显存也要留给后面的 KV Cache，过度 batch 会把自己卡死。
2. Token 阶段：
 1. 每次只处理少量（甚至 1 个）token，几乎没有并行度；
 2. 大部分时间都在访问 KV Cache，彻底的 **memory bandwidth / capacity bound**；

3. 即使用“连续 batch”等调度技巧，绝大多数时间 batch size 也很小（比如只激活 20 个 token），GPU 算力严重浪费。

3. E2E 延迟的绝大部分时间都在 Token 阶段，而不是 Prompt 阶段。

他们怎么破？——“连接旧知识”的视角

课堂上老师讲流水线的时候，最爱说一句话：

“流水线性能由最慢的那一级决定，要尽量平衡各级延迟。”

但是我们以前平衡的是 IF/ID/EX/MEM/WB 这种同一颗 CPU 里的微观阶段。Splitwise 干的是：

把整个 LLM 推理拆成两大“宏观阶段”（Prompt / Token），分别跑在不同机器、甚至不同类型的 GPU 上——这不就是一种跨机器的“异构流水线”设计吗？

具体来说：

1. 阶段划分

1. Prompt 阶段放在一组高性能新 GPU（比如 H100），把算力吃满；
2. Token 阶段放在另一组 GPU 上，可以是算力较弱但带宽还行的老 GPU，因为这里已经不是 compute-bound 了。

2. 流水线寄存器 = KV Cache + 网络

1. Prompt 阶段跑完之后，会生成上下文的 KV Cache；
2. 这些 KV Cache 通过数据中心里的 InfiniBand 背板网络（25–50 GB/s 级别）传给 Token 阶段所在的机器；
3. 这就像在两级流水线之间加了一个巨大的“寄存器”，只是这个寄存器跨了整条机架。

3. 调度和优化目标

1. 在集群规模上，他们不再追求“每块 GPU 的 IPC 最大”，而是追求：
 1. 性能/成本（Perf/\$）最大
 2. 性能/功耗（Perf/W）最大
2. 利用不同 GPU 世代在算力与带宽上的失衡，把“老 GPU”当成 Decode 阶段的“带宽黑洞”，而把“新 GPU”优先用在 Prompt 阶段这种算力密集部分。

实验结果：在真实 trace 下，Splitwise 带来的集群设计可以实现：

1. 在相同成本下吞吐提升 **2.35×**，或者
2. 在吞吐提升 **1.4×** 的同时成本下降 20%。

我看的时候脑子里冒出的联想是：

1. 这跟我们学的“流水线平衡 + 异构多核 (big.LITTLE)”是一个精神：
 1. big core (H100) 干短、算力密集的前端工作 (Prompt)；
 2. little core (老 GPU) 慢慢啃带宽密集的后台任务 (Token)。
2. 区别只是：
 1. 老师画的是一条 CPU 内部流水线；
 2. Splitwise 画的是一整片 GPU 集群的宏观流水线。

经典概念没有死，只是坐到了数据中心的指挥席位上。

2.2 ALISA：把 KV Cache 变成“会思考的 Cache”

问题背景：KV Cache 让计算变快，却把自己变成了“内存炸弹”

我们上课学过：

1. Cache 的目标是用较小、较快的存储层覆盖大部分访问，通过时间局部性/空间局部性来提高命中率。
2. 但在 Transformer 里，KV Cache 有点反传统：它不是靠局部性，而是靠“把所有历史 token 的 KV 都存起来”来避免重复算 Attention——结果就是显存压力爆炸。

ALISA 的论文从一个很实在的角度出发：

1. 在单 GPU + CPU 系统上（比如一张 V100 + 主机内存），LLM 推理已经由内存主导，而不是算力主导；特别是长序列时：
 1. KV Cache 的线性增长直接导致显存 OOM；
 2. 把 KV Cache offload 到 CPU / SSD 又会让 PCIe I/O 变成新的瓶颈。[\(ar5iv\)](#)

关键观察：Attention 权重高度稀疏，且“大模型更稀疏”

ALISA 的一个洞见是：不是什么 token 都值得“终身保留”在 KV Cache 里。

通过分析多种 LLM 的 Attention 权重，他们发现：[\(ar5iv\)](#)

1. 对于生成一个新 token，真正贡献大的只有少数几个历史 token；
2. Attention 权重矩阵整体呈现高度稀疏，且模型越大稀疏性越强；
3. 这就意味着：我们可以在算法上只保留“重要 token”的 KV，其他 token 的 KV 是可以被认为是“死数据”的。

算法层：Sparse Window Attention (SWA) = 更聪明的“Cache Line 选择”

SWA 做的事情，可以类比为：

给每个 token 一个“未来重要性评分”，只给高分 token 分配 KV Cache 的显存空间。

具体来说（简化版本）：

1. 局部静态 + 全局动态稀疏模式

1. 局部静态：保留最近窗口内的一些 token（类似“时间局部性”）。
2. 全局动态：在更长的历史里，根据跨多个 decoding step 的累计 Attention 权重，找出真正重要的 token（“跨步重要性”）。(Zhihu)

2. Token 级别的稀疏掩码

1. 设置一个 Cache Ratio（比如只保留 top 10% 的 token）；
2. 对每一步，把不重要的 token 标记为“稀疏”，它们对应的 KV 不再保留或加载；
3. 用 gather 把剩下的 KV token 重新打包成连续的 dense 张量，保证 GPU 上还是可以用高效的 dense kernel 运行。(ar5iv)

这让我直接想到课上学的：

1. **传统 Cache**：依赖 LRU / LFU 这些简单启发式，隐式假设“最近用过的、空间邻近的会再用到”。
2. **ALISA 的 SWA**：直接读模型自己的 Attention，等于有了一个“预测未来访问的 Oracle”，专门识别真正的“热数据 token”和“冷数据 token”——

这不就是一种“Attention 驱动的 Dead Block Prediction + 自适应 Cache 分配”吗？

系统层：三阶段动态调度 = “会切换策略的分层 Cache + 重新计算”

算法有了稀疏模式，还要落到系统上，ALISA 的系统设计提出了一个**三阶段 Token 级动态调度**：(Zhihu)

1. Phase I – GPU Caching

1. 序列较短、KV 还不大时，所有重要 KV 都留在 GPU，只享受 KV Cache 带来的纯收益。

2. Phase II – GPU-CPU Caching

1. 随着序列增长，KV 体积过大，开始把“相对不重要但可能还会用到”的 KV 搬到 CPU 内存，相当于 GPU-CPU 形成了一个两级 KV Cache 层次。

3. Phase III – Recomputation-Caching

1. 再长下去，即使 CPU 也装不下所有 KV，或者从 CPU 拉 KV 的延迟已经不划算了。
2. 这时他们干脆把最早的一批 KV 在 CPU 中删除，需要时再在 GPU 上用计算重算一遍——典型的“算力换带宽”。

为了决定什么时候切 Phase、KV 在 GPU/CPU 之间移动多少，以及何时改成重算，他们把这个问题形式化为一个最小化总执行时间的优化问题，综合考虑：KV 大小、显存/主存容量、带宽、计算 vs 重算时间等，然后用 profiling + 贪心搜索在离线算出配置。(Zhihu)

此外，为了进一步减小 KV 的体积，他们还把 KV 量化到 INT8 (channel-wise 量化)，在几乎不影响精度的情况下，减少一半的带宽和容量压力。(ar5iv)

最终结果：在单 GPU+CPU 的资源受限系统上，ALISA 比 FlexGen 吞吐快最高 3×，比 vLLM 快最高 1.9×，精度损失可以忽略。(ar5iv)

我联想到的课本知识：

1. KV Cache = 特殊的、token 粒度的 Cache，行大小不是“固定字节数的 cache line”，而是“一个 Token 的 KV 向量”。
2. SWA = 利用“语义 + Attention”来构造未来访问模式，类似把最理想的 Belady Optimal 替换，用模型本身的统计特性做了一个可实现版本。(Zhihu)
3. 三阶段调度 =
 1. Phase I：有点像 L1 命中率很高时，所有数据都在 on-chip；
 2. Phase II：开始让 L2 / DRAM 参与进来；
 3. Phase III：干脆引入“以计算换存储”的策略，就像我们在体系结构里讨论的“recompute vs reload”。

换句话说：ALISA 把一个“傻傻装满的 KV Cache”，改造成了一个“会选择重要数据、会换层、会决定何时重算”的智能缓存层次。

2.3 Pre-gated MoE：把 Expert 当成“可分页的内存页”，再加个预取器

MoE 的矛盾：算力省了，内存更炸了

Mixture-of-Experts (MoE) 的初衷很好：

1. 通过门控 (gate function)，每个 token 只激活少量专家 (top-k)，从而计算量随模型容量增加是次线性，在 FLOPs 维度比 dense LLM 省太多了。

但是系统上出现两个大坑：

1. 专家参数太多，内存压力巨大

1. 像 Switch Transformer，参数量可以是 FLOPs 等效 dense T5 的 75 × ；
2. 单卡 GPU 显存根本塞不下，只能做 expert 并行（多 GPU）或者把专家参数 offload 到 CPU / SSD。

2. 专家是稀疏 + 动态激活的

1. 每个 token 激活的专家集合是由 gate 函数在运行时动态决定的；
2. 这意味着：你在算某个 MoE block 之前，根本不知道要用哪些专家的参数。
3. 如果这些专家在 CPU 内存里，就必须先等 gate 算完，再把对应专家从 CPU 搬到 GPU，才能开始算——CPU→GPU 拖了一个巨大的顺序延迟。

之前的一些 MoE-offload 方案，就是老老实实在干这个“先决定专家、再同步搬运”的流程，结果 GPU 一直在等数据，延迟飙升，TCO 巨大。

Pre-gated MoE 的关键想法：把“缺页中断”变成“提前预取”

读到 Pre-gated MoE 的时候，我第一反应是：

这也太像虚拟内存里的 Page Fault + 预取了吧……

他们的招数可以分两层看：

算法层：改 gate 的语义，让“选专家”提前一层

传统 MoE：

1. 第 N 个 MoE block 的 gate，选择的是“在 N 号 block 里要执行哪些专家”。

Pre-gated MoE：

1. 把 gate 改造成“预 gate”：
 1. 第 N 个 block 的 gate 决定的是 N+1 号 block 要用哪些专家。
2. 这样一来，原先那种“这个 block 先选专家，再搬专家再算”的顺序依赖被打断了：
 1. 第 N 层在计算自己那一层的专家时，
 2. 同时已经帮第 N+1 层算好了“未来要用的专家名单”。

这跟课本里讲的“decoupled access/execute”或“预取器”特别像：

1. 以前：地址生成和访存绑死在一起，load 要等地址；
2. 现在：把“专家选择（地址生成）”前移，让“专家迁移（访存）”可以和当前层的计算并行。

系统层：专家参数 = 可分页数据，CPU = 大容量，“预 gate”= 预取器

系统设计上，Pre-gated MoE 做的事情是：

1. 把大部分专家参数放在 CPU 内存里（像把页放在磁盘或大内存里）；
2. GPU 显存只留一小部分“工作集式”的专家缓存；
3. 当第 N 层的 pre-gate 决定了 N+1 层要用哪些专家时，系统就立即开始：
 1. 从 CPU → GPU 异步迁移这些“未来要访问的专家”，
 2. 同时在 GPU 上继续算 N 层当前已经在的专家。

结果就是：

1. CPU→GPU 的专家迁移延迟被“藏”在当前层的专家计算时间里，像极了我们上课学的“预取把 miss 延迟隐藏在在途计算里”；
2. 从 GPU 的角度看，好像专家永远“刚好在那儿”等你用。

实验结果也很亮眼：

1. 端到端性能只比一个理想的“所有专家都在 GPU 显存”的 oracle 方案慢约 **23%**；
2. 同时 GPU 峰值内存占用减少了 **4.2x**，可以在单卡 GPU 上部署更大的 MoE 模型；
3. 模型精度保持甚至略有提升（他们还顺手做了一些训练上的优化）。

这让我直接把 MoE 想成了：

1. 专家 = 虚拟内存里的页（Page）
2. GPU 显存 = 物理内存
3. CPU 内存 / SSD = 后备存储
4. 传统 MoE-offload = 严重 Page Fault 的按需调页系统
5. Pre-gated MoE = 带有高质量预取器的虚拟内存系统，
 1. pre-gate 就是那个根据历史访问预测未来访问页号的预取逻辑。

所以，虚拟内存那一章没白学。只是我们现在换了一批词：

Page Fault → Expert Miss Page Prefetch → Expert Prefetch TLB /
Cache → GPU 上的小专家集

三、融会贯通：从五级流水线到 AI 加速器的“新旧合体”

把这三篇论文串起来，有一种很强的“故事感”：

1. Splitwise：从集群视角看 Memory Wall

1. 它告诉我们：在大规模部署 LLM 时，**prefill / decode** 两个阶段在“**算 vs 存**”上的不对称性极端严重，
2. 并且通过“跨机器的异构流水线 + KV Cache 迁移”来重新平衡这两个阶段的资源配置。

2. ALISA：在单机上把 KV Cache 玩出花

1. 它把 Attention 的稀疏性（Activation Sparsity）转化成 KV Cache 的存储稀疏性，
2. 再用三阶段调度 + INT8 量化，把“内存墙”往后推了一大截。(ar5iv)

3. Pre-gated MoE：把模型参数本身当成“要分页管理的大数据”

1. 它通过 pre-gating 打破“选专家 → 搬专家 → 算专家”的串行依赖，
2. 用预取 + CPU 存储大专家池的方式，让 MoE 这一大模型形态在单 GPU 上变得可行。

如果用课堂上的话来总结，就是：

摩尔定律放缓之后，体系结构工程师不再单纯指望“多来几个 pipeline stage / 多加点乱序缓冲区”，而是开始通过 **软硬协同（algorithm-system/architecture co-design）** 和 **稀疏化（Sparsity）** 来榨干硬件的每一滴性能。

更具体一点：

1. 软硬协同（Co-design）

1. Splitwise 并没有改模型结构，但用真实业务 trace 驱动集群设计，本质上是“系统调度 × 业务特性”的协同。
2. ALISA 把 Attention 的算法（SWA）和 KV Cache 的系统调度、量化策略一起设计。(ar5iv)
3. Pre-gated MoE 更是把 MoE 的 gate 函数重写，为系统级的异步迁移服务。

2. 稀疏化（Sparsity）

1. Splitwise：从某种意义上，是把**时间维度的稀疏性**（prefill 短暂、decode 长尾）和**并发度稀疏性**（decode 时每步只有少数 active token）利用起来，做硬件分工。
2. ALISA：直接利用 Attention 矩阵的高稀疏性，仅为重要 token 分配 KV Cache。
3. Pre-gated MoE：利用模型结构上的稀疏（每个 token 只走少数专家），再加上对“未来专家访问”的预测，把专家迁移做到几乎无感。

从知识图谱上看，我现在脑子里大概是这样一张图：

1. 经典五级流水线 → 多级缓存 → 虚拟存储 / Page Fault / 预取
↓ (映射到 LLM 推理)
2. LLM 的 Prefill / Decode 两段式执行 → KV Cache 作为跨步状态 → MoE 专家作为大粒度“页”
↓ (搭配新技术)
3. 异构 GPU 流水线 (Splitwise) 稀疏感知 KV Cache (ALISA) 预取式 Expert Offload (Pre-gated MoE)

这三篇 ISCA 论文给我的最大冲击感其实不是“细节有多复杂”，而是——

原来我以为已经“老掉牙”的那些体系结构原理，换了一层语义包装之后，依然可以在 LLM 这种超级现代的工作负载上发挥关键作用。

写完这份调研笔记，我有一种很微妙的满足感：

1. 课上学的经典五级流水线、Cache、分支预测、虚拟内存、预取器、重算 vs 访存的权衡，
2. 并没有停留在纸面，而是顺着 Splitwise / ALISA / Pre-gated MoE 这些工作，真的延伸到了“现代 AI 加速器和推理集群”的设计里。

原本我只是想完成一个“前沿调研”的作业，现在感觉自己从“会背课本上的五级流水线图”升级到了“能用这张图去理解 ISCA 2024 的 LLM 加速器”——这一刻，真的是那种：

“原来体系结构这门课，根本没过时，只是我之前看得太浅了。”